0215. Kth Largest Element in an Array

egory: Heap / Priority Queue | LeetCode: https://leetcode.com/problems/kth-large

Problem

- Return the k-th largest element (in sorted order, duplicates count) without fully sorting, ideally.

Intuition

- Min-heap of size k: O(n log k).
- Quickselect: partition around a pivot; the pivot lands in its final sorted position; recurse only into the side containing index n-k. O(n) average.

Brute Force

- Sort and index: O(n log n).

Optimal Approach

- Target = n - k (index in ascending order).
- Quickselect with a random pivot and Lomuto partition, narrowing [lo, hi] until the pivot index == target.

Worked Example

```
[3,2,1,5,6,4], k = 2 -> target index 4.

pivot 4 -> [3,2,1,4,6,5] pivot at 3 < 4 -> search right [6,5]
pivot 5 -> [..,5,6] pivot at 4 == target -> answer 5
```

Java Solution

```java
public class KthLargestElementInArray {
    private final Random rnd = new Random(42);

    public int findKthLargest(int[] nums, int k) {
        int target = nums.length - k, lo = 0, hi = nums.length - 1;
        while (true) {
            int p = partition(nums, lo, hi);
            if (p == target) return nums[p];
            if (p < target) lo = p + 1;
            else hi = p - 1;
        }
    }

    private int partition(int[] a, int lo, int hi) {
        swap(a, lo + rnd.nextInt(hi - lo + 1), hi);
        int pivot = a[hi], store = lo;
        for (int i = lo; i < hi; i++) if (a[i] < pivot) swap(a, i, store++);
        swap(a, store, hi);
        return store;
    }

    private void swap(int[] a, int i, int j) { int t = a[i]; a[i] = a[j]; a[j] = t; }

    public int findKthLargestHeap(int[] nums, int k) {
        PriorityQueue<Integer> pq = new PriorityQueue<>();
        for (int x : nums) { pq.offer(x); if (pq.size() > k) pq.poll(); }
        return pq.peek();
    }
}
```

Complexity

- Quickselect: O(n) average, O(n^2) worst (random pivot makes it unlikely). Space O(1).
- Heap: O(n log k), O(k).

Pitfalls

- A fixed pivot on sorted input degrades to O(n^2); randomise.